

hw4

李毅PB22051031

1、为什么一般情况下对离散图像的直方图均衡并不能产生完全平坦的直方图？

直方图是PDF的近似，在这个过程中不允许产生新的灰度级，因此在使用上述方法的直方图均衡化应用中，很少出现完全平坦的直方图。灰度级是离散的，同一灰度级的像素不能映射到不同的灰度级上。

2、设已用直方图均衡化技术对一幅图像进行了增强，试证明用这个方法对所得结果增强并不会改变其结果

第一次均衡化后，图像直方图已经达到最大可能的线性化，再次均衡化无法进一步调整，不会有新的合并项，故结果不变。

3、讨论用于空间滤波的平滑滤波器和锐化滤波器的相同点、不同点以及联系。

相同点：都是通过减弱或消除信号傅里叶空间的某些分量来达到增强图像的效果

不同点：平滑滤波器：减弱高频分量，增强低频，平滑图像。锐化滤波器：减弱低频分量，增强高频，锐化图像。

联系：锐化可通过原始图像减去平滑结果得到高频分量，再叠加回原图。平滑对应低通滤波，锐化对应高通滤波，两者频谱特性相反。

4、有一种常用的图像增强技术是将高频增强和直方图均衡化结合起来以达到使边缘锐化的反差增强效果，以上两个操作的先后次序对增强结果有影响吗？为什么？

高频增强是线性操作，直方图均衡是非线性操作。线性操作与非线性操作不可互换次序，故有影响

5、编程实现对lena.bmp分别加入高斯噪声和椒盐噪声，再进行局域平均和中值滤波。

编写代码如下，使用3*3局域平均和中值滤波。

```
import cv2
import numpy as np
import matplotlib.pyplot as plt

# 读取图像（灰度模式）
image = cv2.imread('lena.bmp', 0) # 确保图片路径正确

# 添加高斯噪声
def add_gaussian_noise(img, mean, sigma):
    row, col = img.shape
    gauss = np.random.normal(mean, sigma, (row, col))
    noisy = img + gauss
    noisy = np.clip(noisy, 0, 255)
    return noisy.astype(np.uint8)

# 添加椒盐噪声
def add_salt_pepper_noise(img, prob):
    noisy = img.copy()
    row, col = img.shape
```

```

salt = np.random.rand(row, col) < prob / 2
noisy[salt] = 255
pepper = np.random.rand(row, col) < prob / 2
noisy[pepper] = 0
return noisy

# 生成噪声图像
gaussian_noisy = add_gaussian_noise(image,0,50)
salt_pepper_noisy = add_salt_pepper_noise(image,0.1)

# 均值滤波 (3x3核)
def mean_filter(img, kernel_size=3):
    pad = (kernel_size - 1) // 2
    # 边缘填充 (复制边缘像素)
    padded = np.pad(img, ((pad, pad), (pad, pad)), mode='edge')
    filtered = np.zeros_like(img, dtype=np.float32)
    row,col = img.shape
    for i in range(row):
        for j in range(col):
            window = padded[i:i+kernel_size, j:j+kernel_size]
            filtered[i, j] = np.sum(window)/(kernel_size * kernel_size)
    return filtered.astype(np.uint8)

# 中值滤波 (3x3核)
def median_filter(img, kernel_size=3):
    pad = kernel_size // 2
    # 边缘填充 (复制边缘像素)
    padded = np.pad(img, pad_width=((pad, pad), (pad, pad)), mode='edge')
    filtered = np.zeros_like(img)
    row,col = img.shape
    for i in range(row):
        for j in range(col):
            window = padded[i:i+kernel_size, j:j+kernel_size]
            filtered[i, j] = np.median(window)
    return filtered.astype(np.uint8)

# 对高斯噪声图像滤波
mean_gaussian= mean_filter(gaussian_noisy)
median_gaussian = median_filter(gaussian_noisy)

# 对椒盐噪声图像滤波
mean_sp= mean_filter(salt_pepper_noisy)
median_sp = median_filter(salt_pepper_noisy)

plt.figure(figsize=(12, 8))

plt.subplot(3, 3, 2)
plt.imshow(image, cmap='gray')
plt.title('Original Image')

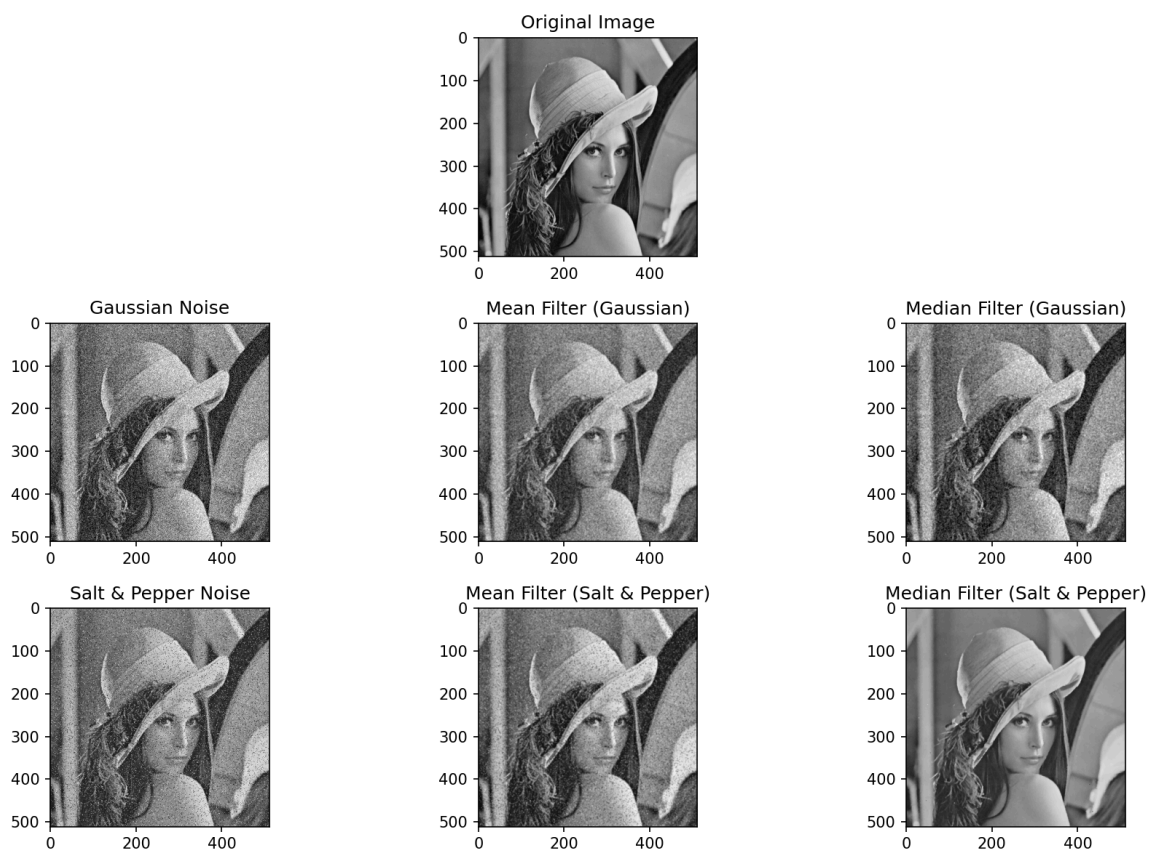
plt.subplot(3, 3, 4)
plt.imshow(gaussian_noisy, cmap='gray')
plt.title('Gaussian Noise')
plt.subplot(3, 3, 5)
plt.imshow(mean_gaussian, cmap='gray')
plt.title('Mean Filter (Gaussian)')

```

```
plt.subplot(3, 3, 6)
plt.imshow(median_gaussian, cmap='gray')
plt.title('Median Filter (Gaussian)')

plt.subplot(3, 3, 7)
plt.imshow(salt_pepper_noisy, cmap='gray')
plt.title('Salt & Pepper Noise')
plt.subplot(3, 3, 8)
plt.imshow(mean_sp, cmap='gray')
plt.title('Mean Filter (Salt & Pepper)')
plt.subplot(3, 3, 9)
plt.imshow(median_sp, cmap='gray')
plt.title('Median Filter (Salt & Pepper)')

plt.tight_layout()
plt.show()
```



可以看到，均值滤波对于高斯噪声效果较好，但是会让图像变得模糊，中值滤波对于椒盐噪声效果较好。